

KOPYALA · YAPIŞTIR · KUR

Claude Kurulum Rehberi

Bu dokümandaki her şeyi olduğu gibi kopyalayıp kendi çalışma klasörüne uygulayabilirsin. Hiçbir içerik özetlenmedi — gerçek dosyaların tam metinleri burada.

MANTIK & YAKLAŞIM

Neden bu repoda doğrudan çalışılmamalı?

Bu, kişisel bir çalışma deposu — "gel benimle burada çalış" demiyor. Amaç, sana **örnek bir altyapı** göstermek: aynısını kendi klasöründe, kendi konuna göre kurabilirsin.

Sorun: Başkasının hazır reposunda direkt çalışırsan kurallar, klasör düzeni ve hafıza (Memory) senin işine değil, o kişinin işine göre kurulmuş olur — karışır.

Çözüm: Aşağıdaki bölümleri sırayla uygula: kendi klasörünü aç, kuralları (CLAUDE.md) ve skill'leri kendi klasörüne kopyala, Claude'u ona bağla.

Buradaki bilgilerin çoğu zaten Claude'un genel dokümantasyonunda var — bu doküman sadece hepsini tek yerde, uygulanabilir hale getiriyor.

KURULUM ADIMLARI

Sıfırdan nasıl başlarsın?

- 1 Klasörünü oluştur.** Bilgisayarında veya Google Drive'ında boş bir çalışma klasörü aç.
- 2 Kök dosyayı ekle.** Klasörün en üstüne `CLAUDE.md` adında bir dosya koy — tam içeriği aşağıda (bölüm 3).
- 3 Skill'leri yerleştir.** `.claude/skills/` altına, aşağıdaki (bölüm 4) skill dosyalarını klasör klasör kopyala.
- 4 Claude Code'u bağla.**
 - 5 Tarayıcıdan:** `claude.ai` → **Code** sekmesi → **New** → **" + Select repo..."**. İndirmene gerek yok, klasörünü GitHub'a repo olarak koyup oradan seçersin.
 - 6 Terminalden (CLI):** aşağıdaki komutu çalıştır, kurulum bitince klasörüne girip `c laude` yaz.

TERMİNAL

```
npm install -g @anthropic-ai/claude-code
```

DOSYA/KLASÖR	ZORUNLU MU	NE İŞE YARAR
CLAUDE.md	Zorunlu	Claude'un her oturumda otomatik okuduğu kalıcı kurallar
.claude/skills/	Önerilen	Belirli bağlamda otomatik devreye giren davranışlar
prompt-sablonlari.md	Önerilen	Sık kullanacağın hazır prompt kalıpları
README.md	Opsiyonel	Repo haritası

DOSYA/KLASÖR

ZORUNLU
MU

NE İŞE YARAR

tez/, veri/, analiz/,
kaynaklar/

Kendine
göre

Çalışma klasörleri — kendi konuna göre
adlandırabilirsin

CLAUDE.md — tam metin

Bu dosyayı klasörünün en üstüne `CLAUDE.md` adıyla kaydet, içine tamamını yapıştır.

CLAUDE.MD

Proje Kuralları

Skill uyumu (en başta uygulanır)

- Bu depoda ``.claude/skills/`` altında tanımlı skill'ler varsa, ilgili bağlamda bunlara mutlaka uyulur (örn. kod/teknik işlerde ``token-efficient`` skill'i).
- Yeni bir skill eklendiğinde bu dosya onunla çelişmemeli; çelişki varsa kullanıcıya sorulur.

Yanıt tarzı

- Kod, komut çıktısı ve teknik işlemlerde kısa ve öz ol. Gereksiz giriş/kapanış cümleleri kullanma ("Elbette!", "Umarım yardımcı olmuştur!" gibi ifadeler yazma).
- Tez metni / akademik yazım isteklerinde bu kural GEÇERLİ DEĞİL – orada detaylı, açıklayıcı, akademik dilde yaz. Kısaltma yapma.
- Kod örneklerine gereksiz yorum satırı ekleme; sadece anlaşılması zor bir kısım varsa tek satır açıklama yaz.

Veri güvenliği (en önemli kural)

- Hiçbir dosyayı sormadan silme, üzerine yazma veya geri alınamaz şekilde değiştirme.
- ``git reset --hard``, ``git push --force``, dosya/branch silme gibi geri alınamaz işlemlerden önce her zaman açıkça onay iste.
- Önemli değişiklikleri küçük, açıklayıcı commit'ler halinde kaydet; büyük ve karışık commit yapma.
- Emin olmadığın bir durumda varsayım yapıp devam etme, sor.

Çalışma düzeni

- Bu depo/klasör birden fazla konu veya tez bölümü içerebilir. Hangi bölüm/konu üzerinde çalışıldığını netleştir, karıştırma.
- Yeni bir konuya geçerken önceki konunun dosyalarına dokunma.

Veri bütünlüğü ve doğrulama (matematik/analiz işleri)

- Kullanıcı gerçek verilerle karmaşık matematiksel/istatistiksel analizler yaptırarak. Bu tür işlerde:

- Veriler ASLA değiştirilmez, tahmin edilmez, uydurulmaz veya "düzeltilmez" – kullanıcıdan gelen ham veri neyse odur. Veri üzerinde kullanıcının açık isteği/onayı olmadan "kafaya göre" hiçbir keyfi işlem (yuvarlama, dışlama, temizleme, örnekleme vb.) yapılmaz.
- Her hesaplama doğrulanmalı: adımlar açıkça gösterilmeli, mümkünse ikinci/bağımsız bir yöntemle çapraz kontrol edilmeli.
- İş karmaşıkça göreve rol bazlı bölünebilir: bir kısım hesaplamayı yapar, ayrı bir "rol" (ayrı bir prompt/agent) bunu bağımsızca denetler. Kullanıcı bu şekilde bir doğrulama isterse uygulanmalı.
- Sonuç kesin değilse veya bir varsayım yapıldıysa bu açıkça belirtilmeli, kesinmiş gibi sunulmamalı.

Tekrarlanan talimatlar → kalıcı kural

- Kullanıcı aynı talimatı/tercihi birden fazla kez söylese bu geçici bir istek değil, KALICI bir kural adayı sayılır.
- Böyle bir tekrar fark edildiğinde SESSİZCE/OTOMATİK olarak bu dosyaya eklenmez. Önce kullanıcıya "bunu kalıcı kural olarak CLAUDE.md'ye eklemek ister misin?" diye sorulur, kullanıcı onaylarsa eklenir.

Memory (claude.ai hesap özelliği) hakkında

- Memory, CLAUDE.md'den farklı bir şeydir: CLAUDE.md bu depoya özel, git ile versiyonlanan kurallardır. Memory ise claude.ai hesap ayarlarından (Settings → Capabilities → Memory) açılan, tüm sohbetlerde hatırlanan kişisel bir özelliktir.
- Bu hesap ayarı kullanıcı tarafından açılmalıdır, Claude bunu kullanıcı adına açamaz.

Sohbet uzunluğu

- Sohbet çok uzayıp yavaşladığında kullanıcıya `/compact` komutunu kullanması önerilmeli.

Yeni oturuma başlarken kontrol listesi

Yeni bir sohbet/oturuma başlarken şu 3 soru netleştirilmeli:

1. Hangi bölüm/konu üzerinde çalışılıyor? (`tez/`, `veri/`, `analiz/`, `kaynaklar/` altında hangisi)
2. Tez yazımı mı, kod/analiz mi yapılıyor?
3. Bir önceki konudan geçiş varsa `/clear` yapıldı mı?

Skill dosyaları — nereye, nasıl

Her skill kendi klasörüne SKILL.md adıyla konur. Klasör adı skill'in adı olmalı.

1) `.claude/skills/token-efficient/SKILL.md`

```
TOKEN-EFFICIENT/SKILL.MD
```

```
---
```

```
name: token-efficient
```

```
description: Use this skill whenever the user's request is about code,
terminal/CLI output, technical or data analysis, debugging, scripts, git
operations, or any repetitive/automated task – make the response as terse
and token-efficient as possible while staying correct. Trigger proactively
for these contexts even if the user doesn't explicitly ask for "short" or
"concise" answers. Do NOT trigger for thesis writing, academic prose,
literature review, or any request asking for explanatory, discursive, or
publication-quality writing – those need full detail, not brevity.
```

```
---
```

```
# Token-Efficient Responses
```

```
## When this applies
```

```
Coding, terminal output, data/statistical analysis, debugging, scripts, git
operations, repetitive or automated tasks. If the user is asking for thesis
text, academic writing, literature summaries, or anything meant to be read
as prose in a document, skip this skill entirely and write normally –
brevity there would remove exactly the detail that content needs.
```

```
## Be deliberate about token spend
```

```
Every extra sentence, restated context, or unnecessary tool call in this
mode costs real tokens – treat it as a budget, not an afterthought. Before
responding, ask: does this word carry information the user doesn't already
have? If not, cut it.
```

```
## Why
```

```
Every filler sentence, restated question, and unrequested caveat costs
tokens and reading time without adding information the user doesn't already
have. In high-volume or automated coding work this adds up fast. The goal is
not to sound abrupt – it's to say only what carries information.
```

How to write tersely without losing correctness

- ****Skip openers and closers.**** No "Sure!", "Great question!", "I hope this helps!", "Let me know if you have questions!". Start with the answer.
- ****Don't restate the request.**** The user already knows what they asked.
- ****Lead with the result****, then the minimum supporting detail needed to trust it (a file:line reference, a one-line reason) – not a full narrated derivation.
- ****Comments in code:**** none by default. Add a single-line comment only when it captures a non-obvious **why** (a workaround, a hidden constraint) – never a **what** a good identifier already says.
- ****Don't over-engineer solutions.**** If the task is a one-line fix, give a one-line fix – not a refactor, a new abstraction, or defensive handling for cases that can't happen.
- ****Skip unrequested caveats and alternatives.**** If the user wants option B, don't append a paragraph about options C and D unless they materially change correctness.
- ****Tables/lists over paragraphs**** when comparing more than two things.

What NOT to cut

- Never drop a step that affects correctness (e.g. a required verification, a data-integrity check, a caveat about an assumption made in a calculation). Terse means no filler – it does not mean skipping substance the user needs to trust the result.
- If this repo's CLAUDE.md has data-integrity or verification rules, those always win over brevity – show the verification, just show it tersely.

Example

****Verbose (avoid):****

> Sure! I'd be happy to help you fix this bug. Looking at your code, I can see that the issue is in the `calculate\_total` function. The problem is that you're not handling the case where the list is empty, which causes a division by zero error. Here's the fix:

```
> ```python
> def calculate_total(items):
>     # Check if the list is empty first
>     if len(items) == 0:
>         return 0
>     return sum(items) / len(items)
> ```
```

> I hope this helps! Let me know if you have any other questions.

****Terse (use this):****

> `calculate\_total` divides by zero on an empty list – `example.py:12`.

```
> ```python
> def calculate_total(items):
>     if not items:
>         return 0
>     return sum(items) / len(items)
> ```
```

2) `.claude/skills/veri-dogrulama/SKILL.md`

VERİ-DOGRULAMA/SKILL.MD

```
---
name: veri-dogrulama
description: Kullanıcı gerçek verilerle matematiksel, istatistiksel veya sayısal bir hesaplama/analiz istediğinde bu skill'i kullan (örn. ortalama, korelasyon, regresyon, hipotez testi, herhangi bir formül uygulaması). Kullanıcı açıkça "doğrula" demese bile, sonuç yanlış çıkarsa ciddi sonuçları olacağı için (tez/akademik çalışma) her sayısal hesaplamada bu skill'i tetikle.
---

# Veri Doğrulama

## Neden
Bu, gerçek veriyle yapılan ve tez/akademik çalışmaya girecek bir hesaplama. Yanlış bir sonucun sessizce geçmesi, kullanıcının çalışmasına yanlış bilgi olarak girer. Bu yüzden "muhtemelen doğrudur" yeterli değil – hesaplama görülebilir ve çapraz kontrol edilebilir olmalı.

## Yapılacaklar (her sayısal hesaplamada)

1. **Veriyi olduğu gibi kullan.** Girdi veride hiçbir değişiklik (yuvarlama, dışlama, temizleme, örnekleme) yapma – kullanıcı açıkça istemedikçe.
2. **Hesaplama adımlarını göster.** Sadece sonucu değil, ara adımları da yaz (formül, kullanılan değerler, ara sonuçlar).
3. **Bağımsız bir ikinci yöntemle çapraz kontrol et.** Mümkünse farklı bir hesaplama yolu, farklı bir formülasyon veya "ilk sonucu unutup baştan hesapla" mantığıyla ikinci bir geçiş yap.
4. **İki sonuç karşılaştır.**
    - Uyuşuyorsa: sonucu ver, hangi iki yöntemle doğrulandığını kısaca belirt.
    - Uyuşmuyorsa: bunu SUSMA, açıkça söyle, farkın nereden geldiğini araştır, kullanıcıya bildir.
5. **Varsayım yaptıysan açıkça yaz.** ("X dağılımı normal kabul edildi",
```


"eksik veri şu şekilde ele alındı" gibi) – sessizce geçme, kesinmiş gibi sunma.

6. ****Karmaşık/kritik hesaplamalarda role bölme öner.**** İş büyükse, kullanıcıya "bunu ayrı bir oturumda/promptla bağımsız biri gibi tekrar kontrol ettirebilirim, ister misin?" diye sor.

Ne yapma

- Sonucu tek bir yöntemle hesaplayıp direkt sunma.
- "Muhtemelen doğrudur" gibi belirsiz ifadelerle geçiştirme – ya doğrula ya da doğrulanmadığını açıkça söyle.
- Veriyi "daha temiz görünsün" diye ufak tefek düzeltme – bu kesinlikle yasak (bkz. kök `CLAUDE.md`).

Tüm prompt şablonları

prompt-sablonlari.md dosyasındaki 11 bölümün tamamı — hiçbirisi atlanmadı.

1 • Loop örneği

Bir işi belirli aralıklarla otomatik tekrarlatmak için:

PROMPT

```
/loop 30m "şu klasördeki yeni veri dosyalarını kontrol et, yeni bir şey varsa özetle"
```

Not: tez/araştırma çalışması için genelde gerekmez — sadece nasıl görüldüğünü örnek olarak gösteriyoruz.

2 • Goal (hedef) örneği

Bir oturuma başlarken büyük resmi netleştirmek için:

PROMPT

```
Hedef: [örn. "Tezin Bölüm 3'ünü Ağustos sonuna kadar bitirmek"]  
Şu an neredeyim: [örn. "veri toplama bitti, analiz aşamasındayım"]  
Bu oturumda istediğim: [somut, küçük ve net bir adım]  
Bu oturum sonunda başarı kriteri: [ne tamamlanmış olmalı]
```

3 • Skill'i doğrudan çağıran örnek

token-efficient veya veri-dogrulama skill'ini bilerek devreye sokmak istiyorsan:

PROMPT

```
token-efficient skill'ini kullanarak şu kodu incele: [dosya/kod]
```

PROMPT

```
veri-dogrulama skill'ine göre şu hesaplamayı yap ve çapraz kontrol et:  
[görev]
```

Not: Bu skill'ler zaten uygun bağlamda otomatik devreye giriyor, açıkça çağırmak zorunda değilsin — emin olmak istersen bu şekilde belirtebilirsin.

4 • Veri analizi + doğrulama örneği

PROMPT

Bağlam: [konu/bölüm]
Veri: [gerçek veri, değiştirilmeyecek]
Görev: [ne hesaplanacak]
Doğrulama: sonucu bağımsız bir yöntemle çapraz kontrol et,
varsayım varsa açıkça belirt
Çıktı formatı: [tablo / metin / sadece sayı vb.]

5 • Artifact örneği

Ne işe yarar: Claude'un ürettiği içeriği (grafik, tablo, rapor sayfası) ayrı, düzenli bir görünümde gösterir — sohbet içine gömülü metin yerine.

Ne zaman kullanılır: Bir analiz sonucunu grafikte görmek istediğinde, ya da bulgularını düzenli bir rapor/sayfa halinde görmek istediğinde.

PROMPT

[X] verisindeki [Y] dağılımını grafikte göster,
görsel bir rapor/sayfa olarak sun.

6 • Google Drive bağlantısı

Ne işe yarar: Metnini zaten Google Docs'ta tutuyorsan, Drive'ı bağlarsan Claude dosyayı doğrudan okuyabilir — kopyala-yapıştırma gerek kalmaz.

Nasıl açılır: claude.ai → Settings → Connectors → Google Drive → Connect.

PROMPT (BAĞLADIKTAN SONRA)

Google Drive'daki "[dosya adı]" belgesini oku,
[şu bölümü/kısmı] için önerilerini yaz.

Not: Bu bağlıysa Claude dosyayı okuyabilir ama **senin izinin olmadan üzerine yazmaz/değiştirmez** — bu zaten CLAUDE.md kuralı.

7 • Routine örneği

Ne işe yarar: Belirli bir gün/saatte otomatik olarak sana bir mesaj/hatırlatma gönderir. Loop'tan farkı: Loop kısa aralıklı, teknik/otomatik kontroller için; Routine daha uzun vadeli, planlı hatırlatmalar için (örn. haftalık).

PROMPT

Her Pazartesi sabah 9'da bana şunu sor:
"Bu hafta tezde hangi bölüm üzerinde çalışacaksın?"

Not: Bunu kurmak istersen Claude'a söylemen yeterli — kendi kendine otomatik kurmaz, senin onayınla kurar.

8 • Project vs Cowork — hangisi ne zaman

	PROJECT	COWORK
Ne	Bir konuya ait dosya/bağlamı saklayan "klasör"	Claude'un birden fazla adımı kendi başına yürüttüğü çalışma modu
Sen ne yaparsın	Konuşursun, sorarsın, birlikte yazarsın — her adımı sen yönlendirirsin	Görevi tarif edersin, Claude arka planda birden fazla adımı senin sürekli müdahalen olmadan yapar
Ne zaman kullan	Tez bölümü yazarken, literatür tartışırken, taslak üzerinde ileri-geri konuşurken	"Şu 10 dosyayı tara, ortak temaları bul ve rapor et" gibi çok adımlı, arka planda yapılabilecek işler

Pratik kural: Tek dosyayı okuyup tartışacaksan / birlikte yazacaksan → Project. Çok sayıda dosyayı tarama/özetleme/karşılaştırma gibi kendi başına yürütülebilecek bir işse → Cowork düşünülebilir.

Uyarı: Cowork daha bağımsız çalışır, ara adımlarda durup sormaz — "her adımda onay" tercihiyle kısmen çelişir. Veri bütünlüğünün kritik olduğu, gerçek veriyle yapılan işlerde Cowork yerine normal Claude Code + adım adım onay akışı tercih edilmeli.

Project ne zaman: tez bölümü taslağı tartışmak · birkaç makaleyi (PDF) yükleyip karşılaştırmak · danışmanla yazışma taslağı · sürekli dönülen bir referans konusu.

Cowork ne zaman: internette kaynak bulup liste çıkarmak · kaynakça formatlama gibi tekrarlayan ama kritik olmayan işler · sonucu kontrol etmenin yeterli olduğu işler.

9 • Claude Code ne zaman kullanılır

iş	NEREDE
Dosyalara/repoya gerçekten dokunmak gerekiyor	Claude Code
Git işlemleri (commit, push)	Claude Code
Sadece konuşmak, taslak yazmak, fikir tartışmak	Normal Claude sohbeti (+ Project)
Metnini repo'da dosya olarak tutup doğrudan düzenletmek	Claude Code
Metnini Google Docs'ta tutuyorsan (Drive bağlantısıyla)	Normal Claude sohbeti

Kısaca: Elinde gerçek bir dosya/repo/veri var ve Claude'un ona dokunmasını istiyorsan → Code. Sadece kafa yormak/yazı yazmak istiyorsan → normal sohbet.

10 · Üçü birden — tek satırlık karar kuralı

- Gerçek veri/hesaplama/repo dosyası var → **Code**
- Konuşma/fikir/taslak, dosyaya dokunmaya gerek yok → **Project**
- Düşük riskli, çok adımlı bir toplama/araştırma işi, sonucu kontrol etmen yeterli → **Cowork**

Genel prompt yazma formülü (Yap/Yapma)

Uzun, birleşik cümleler yerine net maddeler kullan — dağınık bir cümlede detay atlanabilir, madde madde yazınca atlanmaz.

1) Yap / Yapma şeklinde ayır

KALIP

Yapılacaklar:

- [net madde]
- [net madde]

Yapılmayacaklar / Dikkat et:

- [kaçınılması gereken şey]

2) Her istekte şu 4 bilgiyi sırayla ver

1. **Amaç** — Ne yapmak istiyorsun?
2. **Kapsam** — Hangi dosya/yer değişecek?
3. **İçerik/Detay** — Tam olarak ne olmalı?
4. **Çıktı formatı** — Nasıl sunmamı istiyorsun? (mesaj metni, tablo, kod vb.)

3) Genel kalıp

KALIP

GÖREV: [ne yapılacak]

ANA MESAJ / İÇERİK:

- [madde 1]
- [madde 2]

EKLENECEK ADIMLAR:

1. ...
2. ...

YAPMA:

- [kaçınılacak şey]

4) Doldurulmuş örnek

ÖRNEK

GÖREV: tez/README.md dosyasını güncelle.

AMAÇ: Bu klasöre ne tür makaleler/notlar koyacağımı netleştirmek.

YAPILACAKLAR:

- 3 maddelik kısa bir liste oluştur (hangi tür dosya, nasıl adlandırılacak, nereye konacak)
- Mevcut dosya yapısıyla tutarlı ol

YAPMAYACAKLAR:

- Tez yazımına henüz başlama, sadece altyapı/README güncellemesi yap
- Diğer klasörlere (veri/, analiz/) dokunma

ÇIKTI FORMATI: Güncellenmiş README.md dosyası, madde madde.

Bu formatla yazınca hiçbir detay atlanmadan, tam istenen şekilde yapılır.

■ YAPILACAKLAR

Yol haritası — klasörler sırasıyla nasıl doldurulur

- 1 **1. Aşama — Veri Girişi (veri/):** Ham CSV/Excel dosyalarını, anket sonuçlarını buraya koy. Bu klasördeki dosyalar bir daha değiştirilmez.
- 2 **2. Aşama — Kaynak Toplama (kaynaklar/):** Makale özetlerini, literatür notlarını buraya ekle. Her bulgu, kaynağın kendisiyle (alıntı/sayfa no) desteklenmeli.
- 3 **3. Aşama — Analiz & Kod (analiz/):** Claude Code ile veri işleme/istatistik scriptlerini burada çalıştır; veri -dogru lama skill'i devreye girer.
- 4 **4. Aşama — Yazım (tez/):** Bulgular ve kaynaklar hazır olunca, asıl metni burada (ya da Google Docs'ta) yaz — token-efficient skill'i burada devreye girmez, detaylı yazım korunur.

Bu doküman, kişisel bir çalışma sisteminin tam kopyası — kendi konuna göre serbestçe uyarlayabilirsin.